

Frontend Improvements Guide

This document outlines how to transform the React frontend from a functional prototype to a professional, performant application.

Overview

Current state: Functional React UI with good styling but missing performance optimizations, testing, and UX polish. Target state: Production-ready frontend with optimized rendering, comprehensive tests, and professional polish.

TIER 1: CRITICAL (Week 1)

1. Fix Performance - Add React.memo & useCallback

File: `client/src/components/ChatInterface.tsx`

```
import React, { ReactNode } from 'react';
import type { Message } from '../types';

interface ChatInterfaceProps {
  messages: Message[];
  loading: boolean;
  messagesEndRef: React.RefObject<HTMLDivElement>;
}

export default React.memo(function ChatInterface({
  messages,
  loading,
  messagesEndRef,
}: ChatInterfaceProps) {
  return (
    <div className="chat-interface">
      {messages.length === 0 ? (
        <div className="empty-state">
          <h2>Welcome to LangCoach</h2>
          <p>Start a conversation to get personalized English coaching</p>
        </div>
      ) : (
        <section className="messages-list" role="region" aria-label="Chat messages">
          {messages.map((msg) => (
            <article
              key={msg.timestamp}
              className={`message message-${msg.role}`}
              role="article"
            >
              <div className="message-content">{msg.content}</div>
              {msg.role === 'assistant' && msg.audioUrl && (
                <audio
                  src={msg.audioUrl}

```

```

        controls
        aria-label="Coach response audio"
      />
    )}
  </article>
)}
{loading && (
  <div className="message message-loading" aria-live="polite">
    <div className="spinner" aria-label="Coach is thinking...">
      <div></div>
      <div></div>
      <div></div>
    </div>
  </div>
)}
</section>
)}
<div ref={messagesEndRef} aria-hidden="true" />
</div>
);
});

```

File: client/src/App.tsx (Key performance fixes)

```

import { useCallback, useEffect, useRef } from 'react';

export default function App() {
  // ... state declarations ...

  // Optimize scroll behavior
  useEffect(() => {
    const scrollContainer = messagesEndRef.current?.parentElement;
    if (!scrollContainer) return;

    const isAtBottom =
      scrollContainer.scrollHeight - scrollContainer.scrollTop -
      scrollContainer.clientHeight < 100;

    if (isAtBottom && messages.length > 0) {
      messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
    }
  }, [messages[messages.length - 1]?.timestamp]); // Only depend on last message

  // Wrap handler in useCallback to prevent re-renders
  const sendMessage = useCallback(async (text: string) => {
    if (!text.trim()) return;

    const userMessage: Message = {
      role: 'user',
      content: text,
      timestamp: Date.now(),
    };
  });

```

```

};

setMessages((prev) => [...prev, userMessage]);
setLoading((prev) => ({ ...prev, chat: true }));

try {
  const controller = new AbortController();
  const response = await fetch(`${API_URL}/api/chat`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ message: text }),
    signal: controller.signal,
  });

  if (!response.ok) {
    const error = await response.text();
    throw new Error(error || 'Failed to get response');
  }

  const data = await response.json();

  setMessages((prev) => [
    ...prev,
    {
      role: 'assistant',
      content: data.reply,
      timestamp: Date.now(),
    },
  ]);
} catch (error) {
  showToast(
    error instanceof Error ? error.message : 'Failed to send message',
    'error'
  );
} finally {
  setLoading((prev) => ({ ...prev, chat: false }));
}
}, [API_URL, showToast]);

const handleTranscript = useCallback(async () => {
  try {
    setLoading((prev) => ({ ...prev, transcript: true }));

    const markdown = generateMarkdownSummary();
    const blob = new Blob([markdown], { type: 'text/markdown' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `transcript-${new Date().toISOString().split('T')[0]}.md`;
    a.click();
    URL.revokeObjectURL(url);
  }

```

```

    showToast('Transcript downloaded', 'success');
  } catch (error) {
    showToast(
      error instanceof Error ? error.message : 'Failed to download transcript',
      'error'
    );
  } finally {
    setLoading((prev) => ({ ...prev, transcript: false }));
  }
}, [messages, showToast]);

const handleKeyTakeaways = useCallback(async () => {
  try {
    setLoading((prev) => ({ ...prev, takeaways: true }));

    const response = await fetch(`${API_URL}/api/summarize`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ messages }),
    });

    if (!response.ok) throw new Error('Failed to generate takeaways');

    const data = await response.json();
    setMessages((prev) => [
      ...prev,
      {
        role: 'assistant',
        content: data.takeaways.map((t: string) => `• ${t}`).join('\n'),
        timestamp: Date.now(),
      },
    ]);

    showToast('Takeaways generated', 'success');
  } catch (error) {
    showToast(
      error instanceof Error ? error.message : 'Failed to generate takeaways',
      'error'
    );
  } finally {
    setLoading((prev) => ({ ...prev, takeaways: false }));
  }
}, [messages, API_URL, showToast]);

const handleLogout = useCallback(() => {
  if (confirm('Are you sure you want to logout?')) {
    localStorage.removeItem('isAuthenticated');
    setIsAuthenticated(false);
    setMessages([]);
    showToast('Logged out successfully', 'success');
  }
}, [showToast]);

```

```
// ... rest of component ...  
}
```

2. Add AbortController & Cleanup

File: `client/src/App.tsx` (Add at component top)

```
export default function App() {  
  const abortControllerRef = useRef<AbortController | null>(null);  
  
  // Cleanup on unmount  
  useEffect(() => {  
    return () => {  
      abortControllerRef.current?.abort();  
    };  
  }, []);  
  
  // Use in all fetch calls:  
  const sendMessage = useCallback(async (text: string) => {  
    abortControllerRef.current = new AbortController();  
  
    try {  
      const response = await fetch(`${API_URL}/api/chat`, {  
        method: 'POST',  
        headers: { 'Content-Type': 'application/json' },  
        body: JSON.stringify({ message: text }),  
        signal: abortControllerRef.current.signal,  
      });  
      // ... rest of logic  
    } catch (error) {  
      if (error instanceof Error && error.name !== 'AbortError') {  
        showToast(error.message, 'error');  
      }  
    }  
  }, []);  
}
```

3. Consolidate Loading State

File: `client/src/App.tsx`

```
interface LoadingState {  
  chat: boolean;  
  transcript: boolean;  
  takeaways: boolean;  
  auth: boolean;  
}  
  
interface AppState {
```

```

    isAuthenticated: boolean;
    messages: Message[];
    inputMode: 'text' | 'voice';
    theme: 'light' | 'dark';
    loading: LoadingState;
  }

  const [state, setState] = useState<AppState>({
    isAuthenticated: false,
    messages: [],
    inputMode: 'text',
    theme: 'light',
    loading: { chat: false, transcript: false, takeaways: false, auth: false },
  });

  // Helper function to update loading state
  const setLoading = useCallback((key: keyof LoadingState, value: boolean) => {
    setState((prev) => ({
      ...prev,
      loading: { ...prev.loading, [key]: value },
    }));
  }, []);

  // Use in handlers:
  // setLoading('chat', true);
  // setLoading('chat', false);

```

4. Create Shared Type Definitions

Create `client/src/types/index.ts` :

```

export interface Message {
  role: 'user' | 'assistant';
  content: string;
  timestamp: number;
  audioUrl?: string;
}

export interface ChatRequest {
  message: string;
  sessionId?: string;
}

export interface ChatResponse {
  reply: string;
  audioUrl?: string;
}

export interface TranscribeResponse {
  text: string;
}

```

```

export interface SummarizeResponse {
  transcript: string;
  summary: string;
}

export interface TakeawaysResponse {
  takeaways: string[];
}

export interface LoginResponse {
  token: string;
  username?: string;
}

export interface VoiceRecorderState {
  isRecording: boolean;
  time: number;
  transcript: string;
  isProcessing: boolean;
}

```

Then update imports in all files:

```

import type { Message, ChatResponse } from '../types';

```

5. Create Error Toast Component

Create `client/src/components/Toast.tsx` :

```

import {
  useState,
  useCallbck,
  createContext,
  useContext,
  ReactNode,
} from 'react';
import './Toast.css';

interface Toast {
  id: string;
  message: string;
  type: 'error' | 'success' | 'warning' | 'info';
}

interface ToastContextType {
  show: (message: string, type?: Toast['type']) => void;
}

const ToastContext = createContext<ToastContextType | null>(null);

```

```

export function ToastProvider({ children }: { children: ReactNode }) {
  const [toasts, setToasts] = useState<Toast[]>([]);

  const show = useCallback((message: string, type: Toast['type'] = 'info') => {
    const id = Date.now().toString();
    setToasts((prev) => [...prev, { id, message, type }]);

    setTimeout(() => {
      setToasts((prev) => prev.filter((t) => t.id !== id));
    }, 4000);
  }, []);

  return (
    <ToastContext.Provider value={{ show }}>
      {children}
      <div className="toast-container" role="region" aria-label="Notifications"
aria-live="polite">
        {toasts.map((toast) => (
          <div
            key={toast.id}
            className={`toast toast-${toast.type}`}
            role="status"
            aria-atomic="true"
          >
            {toast.message}
          </div>
        ))}
      </div>
    </ToastContext.Provider>
  );
}

export function useToast() {
  const context = useContext(ToastContext);
  if (!context) {
    throw new Error('useToast must be used within ToastProvider');
  }
  return context;
}

```

Create `client/src/components/Toast.css` :

```

.toast-container {
  position: fixed;
  bottom: var(--spacing-lg);
  right: var(--spacing-lg);
  z-index: 1000;
  display: flex;
  flex-direction: column;
  gap: var(--spacing-md);
}

```



```
    max-width: 400px;
}

.toast {
  padding: var(--spacing-md) var(--spacing-lg);
  border-radius: var(--radius-md);
  font-size: 14px;
  animation: slideIn 0.3s ease;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
  word-break: break-word;
}

.toast-error {
  background: #fee2e2;
  color: #991b1b;
  border-left: 4px solid #dc2626;
}

.toast-success {
  background: #dcfce7;
  color: #166534;
  border-left: 4px solid #22c55e;
}

.toast-warning {
  background: #fef3c7;
  color: #92400e;
  border-left: 4px solid #f59e0b;
}

.toast-info {
  background: #dbeafe;
  color: #0c4a6e;
  border-left: 4px solid #0ea5e9;
}

@keyframes slideIn {
  from {
    transform: translateX(400px);
    opacity: 0;
  }
  to {
    transform: translateX(0);
    opacity: 1;
  }
}

@media (max-width: 600px) {
  .toast-container {
    max-width: 90%;
    right: var(--spacing-md);
    left: var(--spacing-md);
  }
}
```

```
}  
}
```

Update `client/src/main.tsx` :

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import App from './App';  
import { ToastProvider } from './components/Toast';  
import './index.css';  
  
ReactDOM.createRoot(document.getElementById('root')!).render(  
  <React.StrictMode>  
    <ToastProvider>  
      <App />  
    </ToastProvider>  
  </React.StrictMode>  
);
```

6. Create Custom Hook for Audio Playback

Create `client/src/hooks/useAudioPlayback.ts` :

```
import { useRef, useCallback, useState } from 'react';  
import { useToast } from '../components/Toast';  
  
interface UseAudioPlaybackOptions {  
  onComplete?: () => void;  
}  
  
export function useAudioPlayback(options: UseAudioPlaybackOptions = {}) {  
  const audioRef = useRef<HTMLAudioElement | null>(null);  
  const abortControllerRef = useRef<AbortController | null>(null);  
  const [isPlaying, setIsPlaying] = useState(false);  
  const { show: showToast } = useToast();  
  
  const play = useCallback(  
    async (text: string) => {  
      try {  
        setIsPlaying(true);  
        abortControllerRef.current = new AbortController();  
  
        const response = await fetch('/api/tts', {  
          method: 'POST',  
          headers: { 'Content-Type': 'application/json' },  
          body: JSON.stringify({ text }),  
          signal: abortControllerRef.current.signal,  
        });  
  
        if (!response.ok) {
```

```

        throw new Error('Text-to-speech failed');
    }

    const blob = await response.blob();
    const url = URL.createObjectURL(blob);

    if (!audioRef.current) {
        audioRef.current = new Audio();
    }

    audioRef.current.src = url;
    await audioRef.current.play();

    audioRef.current.onended = () => {
        URL.revokeObjectURL(url);
        setIsPlaying(false);
        options.onComplete?.();
    };
} catch (err) {
    if (err instanceof Error && err.name !== 'AbortError') {
        showToast(
            err instanceof Error ? err.message : 'Playback failed',
            'error'
        );
    }
    setIsPlaying(false);
}
}, [showToast]
);

const stop = useCallback(() => {
    if (audioRef.current) {
        audioRef.current.pause();
        audioRef.current.currentTime = 0;
    }
    abortControllerRef.current?.abort();
    setIsPlaying(false);
}, []);

return { play, stop, isPlaying };
}

```

TIER 2: HIGH-IMPACT (Week 2)

7. Improve VoiceRecorder Component

Create `client/src/components/VoiceRecorder.tsx` :

```

import { useState, useRef, useCallback } from 'react';
import { useToast } from './Toast';

```

```

import type { Message, VoiceRecorderState, TranscribeResponse } from '../types';
import './VoiceRecorder.css';

interface VoiceRecorderProps {
  onMessage: (message: Message) => void;
  isLoading: boolean;
  onModeChange: (mode: 'text' | 'voice') => void;
}

export default function VoiceRecorder({
  onMessage,
  isLoading,
  onModeChange,
}: VoiceRecorderProps) {
  const [state, setState] = useState<VoiceRecorderState>({
    isRecording: false,
    time: 0,
    transcript: '',
    isProcessing: false,
  });

  const mediaRecorderRef = useRef<MediaRecorder | null>(null);
  const audioChunksRef = useRef<Blob[]>([]);
  const timerRef = useRef<NodeJS.Timeout | null>(null);
  const { show: showToast } = useToast();

  const updateState = useCallback((updates: Partial<VoiceRecorderState>) => {
    setState((prev) => ({ ...prev, ...updates }));
  }, []);

  const startRecording = useCallback(async () => {
    try {
      const stream = await navigator.mediaDevices.getUserMedia({
        audio: { echoCancellation: true, noiseSuppression: true },
      });

      mediaRecorderRef.current = new MediaRecorder(stream);
      audioChunksRef.current = [];

      mediaRecorderRef.current.ondataavailable = (e) => {
        audioChunksRef.current.push(e.data);
      };

      mediaRecorderRef.current.start();
      updateState({ isRecording: true, time: 0, transcript: '' });

      timerRef.current = setInterval(() => {
        setState((prev) => {
          const newTime = prev.time + 1;
          if (newTime >= 60) {
            stopRecording();
            showToast('Recording stopped (60 second limit)', 'warning');
          }
        });
      }, 1000);
    } catch (error) {
      console.error('Error starting recording:', error);
    }
  }, [updateState, showToast]);
}

```

```

    }
    return { ...prev, time: newTime };
  });
}, 1000);
} catch (err) {
  showToast(
    err instanceof Error ? err.message : 'Microphone access denied',
    'error'
  );
  updateState({ isRecording: false });
}
}, [updateState, showToast]);

const stopRecording = useCallback(async () => {
  if (!mediaRecorderRef.current) return;

  mediaRecorderRef.current.stop();
  updateState({ isRecording: false, isProcessing: true });

  if (timerRef.current) {
    clearInterval(timerRef.current);
  }

  mediaRecorderRef.current.onstop = async () => {
    try {
      const audioBlob = new Blob(audioChunksRef.current, {
        type: 'audio/wav',
      });

      const formData = new FormData();
      formData.append('audio', audioBlob);

      const response = await fetch('/api/transcribe', {
        method: 'POST',
        body: formData,
      });

      if (!response.ok) {
        throw new Error('Transcription failed');
      }

      const data: TranscribeResponse = await response.json();

      onMessage({
        role: 'user',
        content: data.text,
        timestamp: Date.now(),
      });

      updateState({
        transcript: data.text,
        isProcessing: false,

```

```

        time: 0,
    });

    showToast('Message sent', 'success');
} catch (err) {
    showToast(
        err instanceof Error ? err.message : 'Failed to process audio',
        'error'
    );
    updateState({ isProcessing: false });
}
};
}, [updateState, onMessage, showToast]);

const formatTime = (seconds: number) => {
    const mins = Math.floor(seconds / 60);
    const secs = seconds % 60;
    return `${mins}:${secs.toString().padStart(2, '0')}`;
};

return (
    <div className="voice-recorder">
        {state.isRecording ? (
            <div className="recording-state">
                <div
                    className="recording-badge"
                    aria-live="polite"
                    aria-label={`Recording: ${formatTime(state.time)}`}
                >
                    <span className="recording-indicator"></span>
                    Recording: {formatTime(state.time)}
                </div>
                <button
                    onClick={stopRecording}
                    className="btn btn-secondary"
                    aria-label="Stop recording"
                >
                    STOP
                </button>
            </div>
        ) : (
            <button
                onClick={startRecording}
                disabled={isLoading || state.isProcessing}
                className="btn btn-primary"
                aria-label="Start voice recording"
                title="Record your message (max 60 seconds)"
            >
                {state.isProcessing ? 'Processing...' : 'SPEAK'}
            </button>
        )}
    </div>

```

```

    {state.transcript && !state.isProcessing && (
      <div className="transcript-preview">
        <label className="transcript-label">Transcript:</label>
        <p className="transcript-text">{state.transcript}</p>
        <button
          onClick={() => updateState({ transcript: '' })}
          className="btn btn-tertiary"
          aria-label="Clear transcript"
        >
          Clear
        </button>
      </div>
    )}
  </div>
);
}

```

Create `client/src/components/VoiceRecorder.css` :

```

.voice-recorder {
  display: flex;
  flex-direction: column;
  gap: var(--spacing-md);
  padding: var(--spacing-md);
  background: var(--color-bg-secondary);
  border-radius: var(--radius-lg);
}

.recording-state {
  display: flex;
  align-items: center;
  gap: var(--spacing-md);
}

.recording-badge {
  display: flex;
  align-items: center;
  gap: var(--spacing-sm);
  font-weight: 500;
  color: var(--color-accent);
  font-size: 14px;
}

.recording-indicator {
  display: inline-block;
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: var(--color-accent);
  animation: pulse 1.5s infinite;
}

```

```

@keyframes pulse {
  0%, 100% {
    opacity: 1;
  }
  50% {
    opacity: 0.3;
  }
}

.transcript-preview {
  padding: var(--spacing-md);
  background: var(--color-bg);
  border-radius: var(--radius-md);
  border-left: 3px solid var(--color-accent);
}

.transcript-label {
  display: block;
  font-size: 12px;
  font-weight: 600;
  color: var(--color-text-secondary);
  margin-bottom: var(--spacing-sm);
  text-transform: uppercase;
  letter-spacing: 0.5px;
}

.transcript-text {
  margin: 0 0 var(--spacing-md) 0;
  line-height: 1.5;
  color: var(--color-text);
  word-break: break-word;
}

```

8. Add Error Boundary Component

Create `client/src/components/ErrorBoundary.tsx` :

```

import React, { ReactNode, ErrorInfo } from 'react';
import './ErrorBoundary.css';

interface Props {
  children: ReactNode;
}

interface State {
  hasError: boolean;
  error?: Error;
}

export class ErrorBoundary extends React.Component<Props, State> {

```



```

constructor(props: Props) {
  super(props);
  this.state = { hasError: false };
}

static getDerivedStateFromError(error: Error): State {
  return { hasError: true, error };
}

componentDidCatch(error: Error, errorInfo: ErrorInfo) {
  console.error('Uncaught error:', error, errorInfo);
}

render() {
  if (this.state.hasError) {
    return (
      <div className="error-boundary">
        <div className="error-content">
          <h2>Something went wrong</h2>
          <p className="error-message">{this.state.error?.message}</p>
          <p className="error-hint">
            Try refreshing the page or contacting support if the problem persists.
          </p>
          <button
            onClick={() => window.location.reload()}
            className="btn btn-primary"
          >
            Reload page
          </button>
        </div>
      </div>
    );
  }

  return this.props.children;
}

```

Create `client/src/components/ErrorBoundary.css` :

```

.error-boundary {
  display: flex;
  align-items: center;
  justify-content: center;
  min-height: 100vh;
  background: var(--color-bg);
  padding: var(--spacing-lg);
}

.error-content {
  text-align: center;
}

```

```

    max-width: 400px;
  }

.error-content h2 {
  color: #dc2626;
  margin-bottom: var(--spacing-md);
}

.error-message {
  color: var(--color-text-secondary);
  margin-bottom: var(--spacing-sm);
  font-size: 14px;
  font-family: monospace;
  background: var(--color-bg-secondary);
  padding: var(--spacing-md);
  border-radius: var(--radius-md);
  overflow-x: auto;
}

.error-hint {
  color: var(--color-text-secondary);
  margin-bottom: var(--spacing-lg);
  font-size: 14px;
}

```

9. Add Spinner Component

Create `client/src/components/Spinner.tsx` :

```

import './Spinner.css';

interface SpinnerProps {
  size?: 'sm' | 'md' | 'lg';
  label?: string;
}

export function Spinner({
  size = 'md',
  label = 'Loading',
}: SpinnerProps) {
  return (
    <div
      className={`spinner spinner-${size}`}
      role="status"
      aria-label={label}
      aria-busy="true"
    >
      <div></div>
      <div></div>
      <div></div>
    </div>
  );
}

```

```
);  
}
```

Create `client/src/components/Spinner.css` :

```
.spinner {  
  display: inline-flex;  
  gap: 4px;  
  align-items: center;  
  justify-content: center;  
}  
  
.spinner div {  
  background: var(--color-accent);  
  border-radius: 50%;  
  animation: bounce 1.4s infinite ease-in-out both;  
}  
  
.spinner-sm div {  
  width: 6px;  
  height: 6px;  
}  
  
.spinner-md div {  
  width: 8px;  
  height: 8px;  
}  
  
.spinner-lg div {  
  width: 12px;  
  height: 12px;  
}  
  
.spinner div:nth-child(1) {  
  animation-delay: -0.32s;  
}  
  
.spinner div:nth-child(2) {  
  animation-delay: -0.16s;  
}  
  
@keyframes bounce {  
  0%, 80%, 100% {  
    transform: scale(0);  
    opacity: 0.5;  
  }  
  40% {  
    transform: scale(1);  
    opacity: 1;  
  }  
}
```

10. Add Component Tests

Create `client/src/components/__tests__/ChatInterface.test.tsx` :

```
import { render, screen } from '@testing-library/react';
import { describe, it, expect } from 'vitest';
import ChatInterface from '../ChatInterface';
import type { Message } from '../../types';

describe('ChatInterface', () => {
  const mockRef = { current: null };

  it('renders empty state when no messages', () => {
    render(
      <ChatInterface
        messages={[]}
        loading={false}
        messagesEndRef={mockRef}
      />
    );
    expect(screen.getByText(/Start a conversation/i)).toBeInTheDocument();
  });

  it('renders user and assistant messages', () => {
    const messages: Message[] = [
      {
        role: 'user',
        content: 'Hello, coach',
        timestamp: 1,
      },
      {
        role: 'assistant',
        content: 'Hello! How can I help you today?',
        timestamp: 2,
      },
    ];

    render(
      <ChatInterface
        messages={messages}
        loading={false}
        messagesEndRef={mockRef}
      />
    );

    expect(screen.getByText('Hello, coach')).toBeInTheDocument();
    expect(screen.getByText('Hello! How can I help you
today?')).toBeInTheDocument();
  });

  it('shows loading indicator when loading', () => {
```

```

    render(
      <ChatInterface
        messages={[]}
        loading={true}
        messagesEndRef={mockRef}
      />
    );
    expect(screen.getByText(/Coach is thinking/i)).toBeInTheDocument();
  });

  it('renders messages with correct roles', () => {
    const messages: Message[] = [
      { role: 'user', content: 'Test', timestamp: 1 },
    ];

    const { container } = render(
      <ChatInterface
        messages={messages}
        loading={false}
        messagesEndRef={mockRef}
      />
    );

    const messageEl = container.querySelector('.message-user');
    expect(messageEl).toBeInTheDocument();
  });
});

```

Create `client/src/components/__tests__/VoiceRecorder.test.tsx` :

```

import { render, screen } from '@testing-library/react';
import { describe, it, expect, vi } from 'vitest';
import VoiceRecorder from '../VoiceRecorder';

// Mock the useToast hook
vi.mock('../components/Toast', () => ({
  useToast: () => ({
    show: vi.fn(),
  }),
}));

describe('VoiceRecorder', () => {
  const mockOnMessage = vi.fn();
  const mockOnModeChange = vi.fn();

  it('renders speak button when not recording', () => {
    render(
      <VoiceRecorder
        onMessage={mockOnMessage}
        isLoading={false}
        onModeChange={mockOnModeChange}
      />
    );
  });
});

```

```

    />
  );

  expect(screen.getByRole('button', { name: /speak/i })).toBeInTheDocument();
});

it('disables speak button when loading', () => {
  render(
    <VoiceRecorder
      onMessage={mockOnMessage}
      isLoading={true}
      onModeChange={mockOnModeChange}
    />
  );

  const button = screen.getByRole('button', { name: /speak/i });
  expect(button).toBeDisabled();
});
});

```

Update `client/package.json` :

```

{
  "scripts": {
    "test": "vitest run",
    "test:watch": "vitest --watch",
    "test:coverage": "vitest run --coverage",
    "test:ui": "vitest --ui"
  },
  "devDependencies": {
    "vitest": "^1.0.0",
    "@vitest/ui": "^1.0.0",
    "@testing-library/react": "^14.0.0",
    "@testing-library/jest-dom": "^6.0.0",
    "jsdom": "^22.0.0"
  }
}

```

TIER 3: POLISH (Week 3)

11. Add Dark Mode Toggle

Update `client/src/App.tsx` :

```

// In component initialization
useEffect(() => {
  const prefersDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
  const stored = localStorage.getItem('theme');
  const theme = stored ? (stored as 'light' | 'dark') : (prefersDark ? 'dark' : 'light');

```

```

    document.documentElement.setAttribute('data-theme', theme);
    setState((prev) => ({ ...prev, theme }));
  }, []);

const toggleTheme = useCallback(() => {
  setState((prev) => {
    const newTheme = prev.theme === 'dark' ? 'light' : 'dark';
    document.documentElement.setAttribute('data-theme', newTheme);
    localStorage.setItem('theme', newTheme);
    return { ...prev, theme: newTheme };
  });
}, []);

// In JSX
<button
  onClick={toggleTheme}
  className="theme-toggle"
  aria-label={`Switch to ${state.theme === 'dark' ? 'light' : 'dark'} mode`}
  title="Toggle dark mode"
>
  {state.theme === 'dark' ? '☀️' : '🌙'}
</button>

```

Update `client/src/App.css` :

```

/* Light mode (default) */
:root {
  --color-bg: #ffffff;
  --color-bg-secondary: #f9fafb;
  --color-text: #111827;
  --color-text-secondary: #6b7280;
  --color-accent: #7c3aed;
  --color-border: #e5e7eb;
}

/* Dark mode via data attribute */
:root[data-theme="dark"] {
  --color-bg: #0f172a;
  --color-bg-secondary: #1e293b;
  --color-text: #f1f5f9;
  --color-text-secondary: #94a3b8;
  --color-accent: #a78bfa;
  --color-border: #334155;
}

/* System preference dark mode */
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --color-bg: #0f172a;
    --color-bg-secondary: #1e293b;
  }
}

```

```

    --color-text: #f1f5f9;
    --color-text-secondary: #94a3b8;
    --color-accent: #a78bfa;
    --color-border: #334155;
  }
}

.theme-toggle {
  background: none;
  border: none;
  font-size: 20px;
  cursor: pointer;
  padding: var(--spacing-sm);
  border-radius: var(--radius-md);
  transition: background 0.2s ease;
}

.theme-toggle:hover {
  background: var(--color-bg-secondary);
}

```

12. Add Message Actions (Copy, Delete)

Create `client/src/components/MessageActions.tsx` :

```

import { useCallback } from 'react';
import { useToast } from './Toast';
import './MessageActions.css';

interface MessageActionsProps {
  content: string;
  onDelete?: () => void;
}

export function MessageActions({
  content,
  onDelete,
}: MessageActionsProps) {
  const { show: showToast } = useToast();

  const copyToClipboard = useCallback(async () => {
    try {
      navigator.clipboard.writeText(content);
      showToast('Copied to clipboard', 'success');
    } catch {
      showToast('Failed to copy', 'error');
    }
  }, [content, showToast]);

  return (
    <div className="message-actions">

```



```

    <button
      onClick={copyToClipboard}
      className="action-btn"
      title="Copy message"
      aria-label="Copy message to clipboard"
    >
      
    </button>
    {onDelete && (
      <button
        onClick={() => {
          if (confirm('Delete this message?')) {
            onDelete();
          }
        }}
        className="action-btn"
        title="Delete message"
        aria-label="Delete message"
      >
        
      </button>
    )}
  </div>
);
}

```

Create `client/src/components/MessageActions.css` :

```

.message-actions {
  display: flex;
  gap: var(--spacing-sm);
  opacity: 0;
  transition: opacity 0.2s ease;
}

.message:hover .message-actions {
  opacity: 1;
}

.action-btn {
  background: none;
  border: none;
  font-size: 16px;
  cursor: pointer;
  padding: 4px 8px;
  border-radius: 4px;
  transition: background 0.2s ease;
}

.action-btn:hover {
  background: var(--color-border);
}

```

```

}

.action-btn:active {
  transform: scale(0.95);
}

```

13. Add Conversation Export

Create `client/src/hooks/useConversationExport.ts` :

```

import { useCallback } from 'react';
import type { Message } from '../types';

export function useConversationExport() {
  const exportAsJSON = useCallback((messages: Message[], filename?: string) => {
    const data = {
      exportDate: new Date().toISOString(),
      messageCount: messages.length,
      messages,
    };

    const blob = new Blob([JSON.stringify(data, null, 2)], {
      type: 'application/json',
    });

    downloadFile(blob, filename || `conversation-${Date.now()}.json`);
  }, []);

  const exportAsMarkdown = useCallback((messages: Message[], filename?: string) => {
    let md = '# English Coaching Session\n\n';
    md += `**Date:** ${new Date().toLocaleString()}\n`;
    md += `**Messages:** ${messages.length}\n\n`;
    md += '---\n\n';

    messages.forEach((msg, i) => {
      md += `### Message ${i + 1}\n\n`;
      md += `**${msg.role === 'user' ? 'You' : 'Coach'}:**\n\n`;
      md += `${msg.content}\n\n`;
    });

    const blob = new Blob([md], { type: 'text/markdown' });
    downloadFile(blob, filename || `conversation-${Date.now()}.md`);
  }, []);

  const exportAsCSV = useCallback((messages: Message[], filename?: string) => {
    const headers = ['Timestamp', 'Role', 'Message'];
    const rows = messages.map((msg) => [
      new Date(msg.timestamp).toISOString(),
      msg.role,
      `${msg.content.replace(/"/g, '""')}`
    ]);
  }, []);
}

```

```

const csv = [
  headers.join(','),
  ...rows.map((row) => row.join(',')),
].join('\n');

const blob = new Blob([csv], { type: 'text/csv' });
downloadFile(blob, filename || `conversation-${Date.now()}.csv`);
}, []);

return { exportAsJSON, exportAsMarkdown, exportAsCSV };
}

function downloadFile(blob: Blob, filename: string) {
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
  URL.revokeObjectURL(url);
}

```

TESTING SETUP

Create `vitest.config.ts` in client directory:

```

import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';
import path from 'path';

export default defineConfig({
  plugins: [react()],
  test: {
    globals: true,
    environment: 'jsdom',
    setupFiles: ['./src/test/setup.ts'],
    coverage: {
      provider: 'v8',
      reporter: ['text', 'json', 'html'],
      exclude: [
        'node_modules/',
        'src/test/',
      ],
    },
  },
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),
    },
  },
});

```

```
    },  
  },  
});
```

Create `client/src/test/setup.ts` :

```
import '@testing-library/jest-dom';  
import { expect, afterEach, vi } from 'vitest';  
import { cleanup } from '@testing-library/react';  
  
afterEach(() => {  
  cleanup();  
});  
  
// Mock window.matchMedia  
Object.defineProperty(window, 'matchMedia', {  
  writable: true,  
  value: vi.fn().mockImplementation(query => ({  
    matches: false,  
    media: query,  
    onchange: null,  
    addListener: vi.fn(),  
    removeListener: vi.fn(),  
    addEventListener: vi.fn(),  
    removeEventListener: vi.fn(),  
    dispatchEvent: vi.fn(),  
  })),  
});
```

QUICK CHECKLIST

- ☐ Add React.memo to ChatInterface
- ☐ Add useCallback to all handlers
- ☐ Add AbortController to fetch calls
- ☐ Consolidate loading state
- ☐ Extract shared types
- ☐ Create Toast component
- ☐ Create useAudioPlayback hook
- ☐ Improve VoiceRecorder with better state
- ☐ Add ErrorBoundary
- ☐ Add Spinner component
- ☐ Write component tests (>80% coverage)
- ☐ Add ESLint config
- ☐ Add dark mode toggle
- ☐ Add message copy/delete actions
- ☐ Add export functionality
- ☐ Update accessibility (ARIA labels)

- ☐ Run `npm run test` and verify all pass
 - ☐ Run `npm run lint` and fix all issues
 - ☐ Test in production build with `npm run build`
-

PERFORMANCE TARGETS

After implementation, aim for:

- **Lighthouse scores:** >90 Performance, >95 Accessibility
 - **Time to Interactive:** <2 seconds
 - **Largest Contentful Paint:** <1.5 seconds
 - **Cumulative Layout Shift:** <0.1
 - **Test coverage:** >80% of components
 - **Bundle size:** <150KB (gzipped)
-

ACCESSIBILITY CHECKLIST

- ☐ All interactive elements keyboard accessible
 - ☐ Proper semantic HTML (section, article, etc.)
 - ☐ ARIA labels on buttons/inputs
 - ☐ Color contrast ratios verified
 - ☐ Loading states announced via aria-live
 - ☐ Error messages associated with inputs
 - ☐ Focus visible on all interactive elements
 - ☐ No keyboard traps
 - ☐ Animations respect prefers-reduced-motion
-

Next: After implementing all improvements, follow `BACKEND_IMPROVEMENTS.md` for API enhancements.